

Distribución de calificación

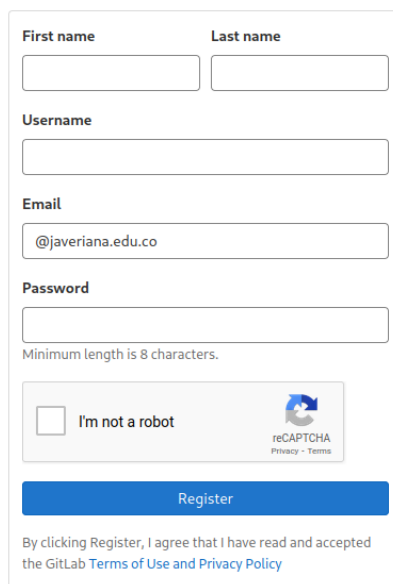
Objetivo	Valor
Refinamiento 1 en archivo <code>README.md</code> - tag: R1	1,0
Refinamiento 2 en archivo <code>README.md</code> - tag: R2	1,5
Código C en el archivo <code>calc.c</code> - tag: cod	2,5

Nota: se considerará como entrega la interacción con el repositorio de código **antes del 5 de febrero de 2022 a las 9 a.m.**

Problema: una calculadora sencilla

Escriba un programa que le pida al usuario que ingrese dos números, los obtenga del usuario e imprima su suma, producto, diferencia, cociente y residuo de la división.

0. Crea una cuenta en GitLab



The image shows the GitLab registration form. It includes input fields for 'First name', 'Last name', 'Username', 'Email' (with '@javeriana.edu.co' pre-filled), and 'Password'. Below the password field is a note: 'Minimum length is 8 characters.' There is a checkbox for 'I'm not a robot' and a reCAPTCHA logo. At the bottom is a blue 'Register' button. Below the button, it says: 'By clicking Register, I agree that I have read and accepted the GitLab [Terms of Use](#) and [Privacy Policy](#)'.

Figura 1: Formulario de creación de cuenta

GitLab es una plataforma, similar a *GitHub*, basada en el conjunto de prácticas llamada *DevOps*[1]. En su versión gratuita, permite almacenamiento de código con control de versiones (*git*). Al completar esta sección, habrás creado una cuenta asociada a tu correo javeriano:

1. Ingresa desde un navegador web a la URL https://gitlab.com/users/sign_up y completa el formulario de creación de cuenta **usando tu correo electrónico javeriano** en el campo *Email*. Se recomienda coincidir el usuario javeriano con el de esta plataforma (según disponibilidad de usuario).

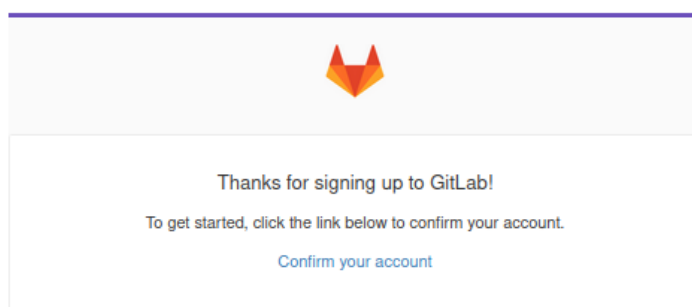


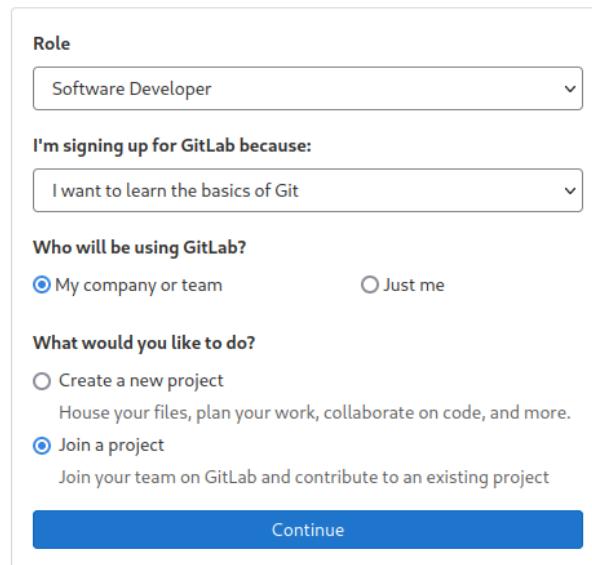
Figura 2: Correo electrónico de verificación

2. Deberás recibir un correo electrónico de verificación a tu cuenta javeriana; haz clic sobre el enlace [Confirm your account](#). Esto abrirá una nueva pestaña en tu navegador web. **Ignora el error** y procede a iniciar sesión.

Figura 3: Inicio de sesión

3. Para evitar confusiones, cierra las pestañas del navegador anteriores e ingresa a la URL https://gitlab.com/users/sign_in. Usa las credenciales que acabas de definir en el paso 1.

Nota: cuando uses un computador con usuario compartido (Ej.: laboratorio), no uses la opción [Remember me](#) ni aceptes guardar credenciales en el navegador.



The image shows a sign-up survey form for GitLab. It contains the following sections:

- Role:** A dropdown menu with "Software Developer" selected.
- I'm signing up for GitLab because:** A dropdown menu with "I want to learn the basics of Git" selected.
- Who will be using GitLab?:** Two radio buttons: "My company or team" (selected) and "Just me".
- What would you like to do?:** Two radio buttons: "Create a new project" (unselected) and "Join a project" (selected). Below "Join a project" is the text "Join your team on GitLab and contribute to an existing project".
- Continue:** A blue button at the bottom.

Figura 4: Encuesta de personalización

4. Completa la encuesta de personalización con los siguientes valores:

- *Role:* Software Developer
- *I'm signing up for GitLab because:* I want to learn the basics of Git
- *Who will be using Gitlab?:* My company or team
- *What would you like to do?:* Join a project

Luego, clic en [Continue](#).

1. Crea un grupo de proyectos para la asignatura

En esta secuencia de pasos se creará el grupo[2] que contendrá el código de todos los **talleres individuales** que realizaremos durante el semestre:

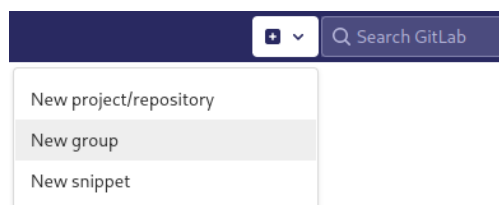


Figura 5: Crea un grupo de proyectos

1. En la barra superior, clic sobre el símbolo + cerca al cuadro de búsqueda y luego clic sobre [New group](#).

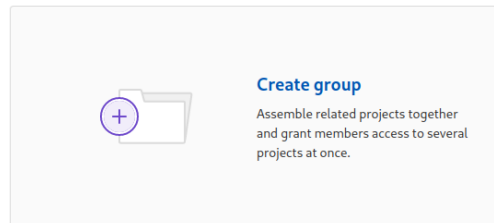


Figura 6: Confirma creación de grupo

2. Confirma tu decisión haciendo clic sobre el cuadro [Create group](#).

New group › **Create group**

Group name

Group URL

Visibility level

Figura 7: Asigna el nombre del grupo

3. Escriba el nombre de grupo asignado en listado adjunto a este enunciado en el cuadro de texto *Group name*, automáticamente se completará la URL¹ en el cuadro de texto *Group URL* (**no modifique** el valor asignado).

Visibility level

Who will be able to see this group? [View the documentation](#)

☒ **Private**
The group and its projects can only be viewed by members.

☐ **Public**
The group and any public projects can be viewed without any authentication.

Figura 8: Define la visibilidad del grupo

4. Selecciona [Private](#) en la sección *Visibility level* del formulario.

¹Usa esta URL para acceder directamente a tu grupo de proyectos guardándola como marcador en el navegador web de tu computador personal.

Now, personalize your GitLab experience

We'll use this to help surface the right features and information to you.

Role

Software Developer ▼

Who will be using this group?

☒ My company or team ☐ Just me

What will you use this group for?

I want to learn the basics of Git ▼

Invite Members (optional)

Figura 9: Personalización del grupo

5. Responde las preguntas de personalización con los siguientes valores:

- *Role:* Software Developer
- *Who will be using Gitlab?:* My company or team
- *What will you use this group for?:* I want to learn the basics of Git

Invite Members (optional)

Invited users will be added with developer level permissions. [View the documentation](#) to see how to change this later.

Email 1

@javeriana.edu.co

Email 2

@javeriana.edu.co

+ Invite another member

Create group

Cancel

Figura 10: Invitación a miembros

6. En el campo *Email 1* agrega el correo electrónico del profesor.

Nota: es posible agregar o modificar miembros del grupo posteriormente[3].

7. Clic en [Invite another member](#) y agrega en el campo *Email 2* el correo electrónico del monitor.

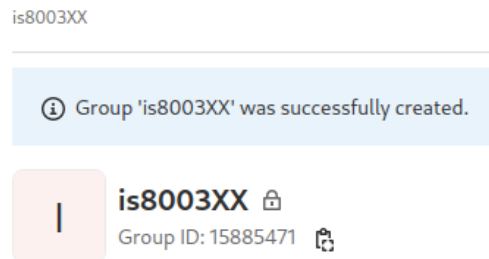


Figura 11: Grupo creado

8. Por último clic en el botón [Create group](#). Deberá mostrarse a continuación el sitio principal del grupo que acabas de crear. Siempre podrás encontrar los grupos a los que perteneces en la ruta [Menu > Groups > Your groups](#). Adicionalmente, recibirás un correo electrónico notificando asignación de acceso.

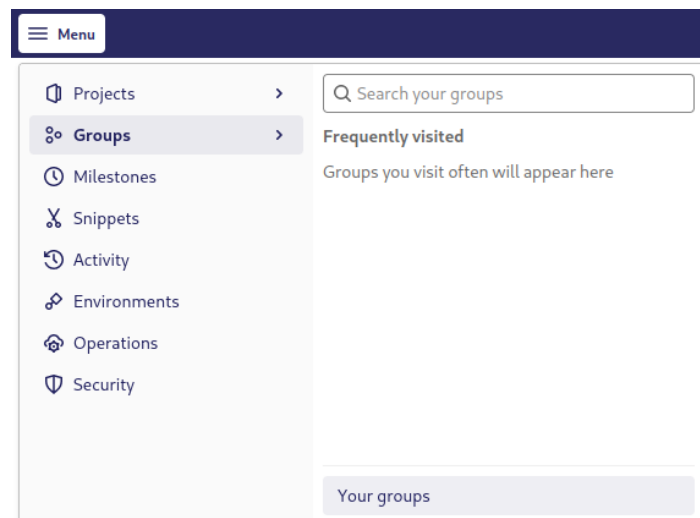


Figura 12: Accediendo a lista de grupos

2. Crea el proyecto: Taller00

Ahora crearás un proyecto[4] donde quedará almacenado el código (y pseudo-código) del problema propuesto.

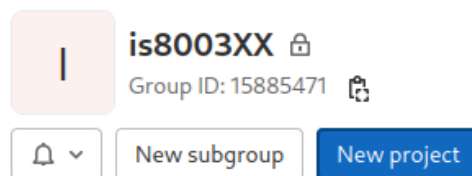


Figura 13: Crea un proyecto

1. En la página inicial de tu grupo de proyectos, clic sobre el botón **New project**.

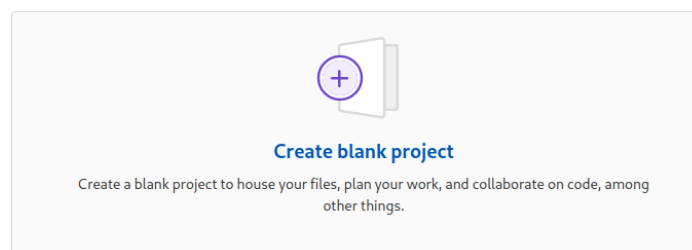


Figura 14: Selecciona proyecto en blanco

2. Clic sobre la opción **Create blank project**.

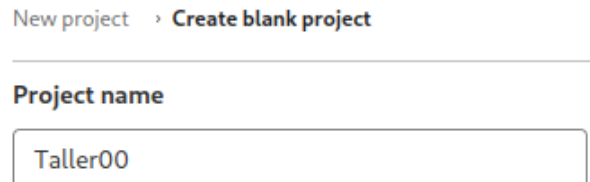


Figura 15: Asigna el nombre del proyecto

3. Asigna el nombre **Taller00** en el cuadro de texto **Project name**. Deja los valores de campos *Project URL* y *Project slug* por defecto.

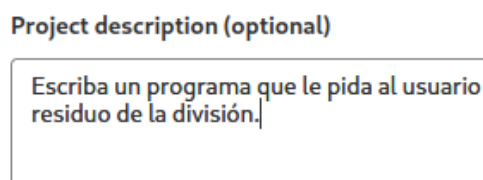


Figura 16: Copia el enunciado del problema

4. En el cuadro de texto *Project description* **copia el enunciado** del problema.

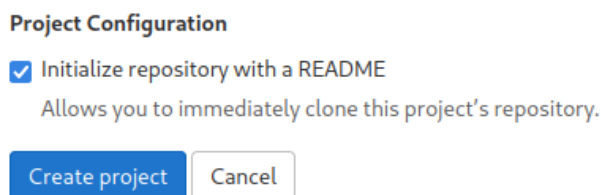


Figura 17: Crea el proyecto

5. Mantén la opción *Initialize repository with a README* activa y haz clic sobre el botón **Create project**.

4. Desactiva el *Administrador de Credenciales de Windows* para Git

Nota: este paso lo puedes omitir si vas a trabajar desde tu computador personal. En equipos del campus universitario **no almacenes credenciales**.

Como precaución de seguridad, con las siguientes instrucciones evitarás que las credenciales para acceder al repositorio queden almacenadas en el *Administrador de Credenciales* de Windows.

1. Abre una ventana de **PowerShell** con permisos administrativos:

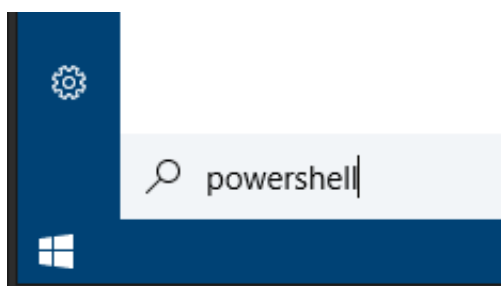


Figura 18: Busca **powershell** en el *Menú Inicio*.

- a. Oprime tecla Windows para abrir el *Menú Inicio*.
- b. Escribe **powershell** en el cuadro de búsqueda.

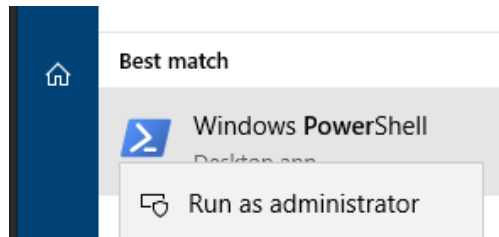


Figura 19: Ejecuta `powershell` como administrador.

- c. Haz clic derecho sobre el mejor resultado de búsqueda.
 - d. **Clic sobre opción Ejecutar como Administrador.**
 - e. Clic sobre botón **Sí** en la ventana emergente.
2. Ejecuta el siguiente comando en la ventana de `PowerShell` que acabas de abrir:

```
1 PS > git config --system --unset credential.helper manager
```

3. **Cierra** la ventana de `PowerShell`.

5. Clona el repositorio

Nota: tendrás que clonar tu repositorio cada vez que decidas trabajar sobre tu código desde un computador sin una copia local.

En esta secuencia de pasos obtendrás una copia local del repositorio[5], una de las funcionalidades del proyecto que acabas de crear:

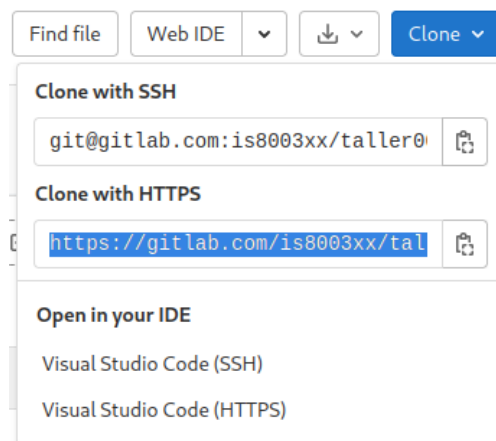


Figura 20: Copia enlace de repositorio.

1. En la página principal del proyecto haz clic sobre la lista desplegable **Clone** y copia la URL ofrecida en cuadro de texto *Clone with HTTPS*.

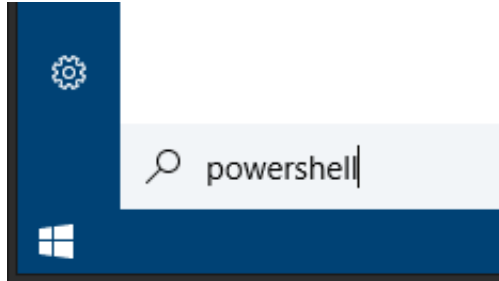


Figura 21: Abre consola para comandos git.

2. Oprime tecla Windows, escribe **powershell** y pulsa la tecla intro. Esto abrirá una nueva ventana de **PowerShell** **sin** permisos administrativos.
3. Cambia de directorio con el comando **cd** a la carpeta (preexistente) que contendrá tu código. Ejemplo:

```
1 PS C:\Users\Administrator> cd Documents\introProg
2 PS C:\Users\Administrator\Documents\introProg>
```

Nota: Siempre podrás verificar en qué carpeta te encuentras usando el comando **pwd**.

4. Ejecuta el comando **git clone URL**, reemplazando **URL** por el enlace de tu repositorio, para obtener una copia local. Ejemplo:

```
1 PS > git clone https://gitlab.com/is8003xx/taller00.git
2 Cloning into 'taller00'...
3 Username for 'https://gitlab.com': g.cristancho01
4 Password for 'https://g.cristancho01@gitlab.com':
5 remote: Enumerating objects: 6, done.
6 remote: Counting objects: 100% (6/6), done.
7 remote: Compressing objects: 100% (4/4), done.
8 remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
9 Receiving objects: 100% (6/6), done.
```

Git te preguntará el usuario y contraseña que definiste al inicio de este taller.

Nota: mientras escribas tu contraseña, **git** no mostrará avance de cursor en la terminal. Si tienes dudas de haber escrito la credencial correcta, puedes borrar todos los caracteres ingresados con el atajo de teclado **CTRL+U**.

Si verificas el contenido de la carpeta creada con el nombre del proyecto, encontrarás la carpeta

oculta `.git`²[6] y el archivo `README.md`.

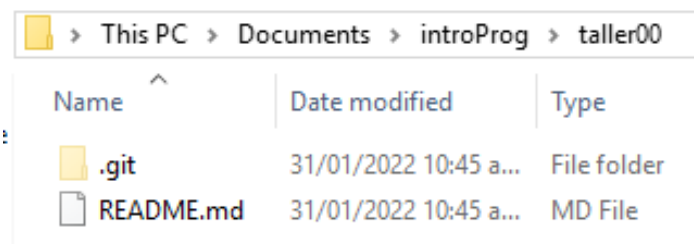


Figura 22: Contenidos de copia local

6. Define tu identidad

En la ventana que usaste para clonar el repositorio, ejecuta los siguientes comandos para definir tu identidad:

```
1 git config --global user.name "Pepito Perez"
2 git config --global user.email pepito@javeriana.edu.co
```

Nota: Este paso debe realizarse cada vez que uses **un computador por primera vez**.

7. Registrando cambios con Git[7]

Git trata los archivos de tu carpeta de trabajo de dos maneras:

- *Tracked*: git **rastrea** los cambios realizados sobre el archivo.
- *Untracked*: git **ignora** los cambios realizados sobre el archivo.

Ahora, los archivos que son rastreados (*tracked*) por git, pueden tener los siguientes estados:

- *Unmodified*: el contenido del archivo coincide con la “instantánea” generada (*commit*).
- *Modified*: el contenido del archivo está en proceso de edición y por tanto **no** coincide con la “instantánea” de referencia.
- *Staged*: consideramos que hemos finalizado con la edición del archivo y **preparamos** el archivo para crear una “instantánea” (*commit*).

²**Nunca borres esta carpeta** mientras estés en medio de ediciones que no han sido enviadas al repositorio remoto, pues contiene toda la información relativa a tu repositorio.

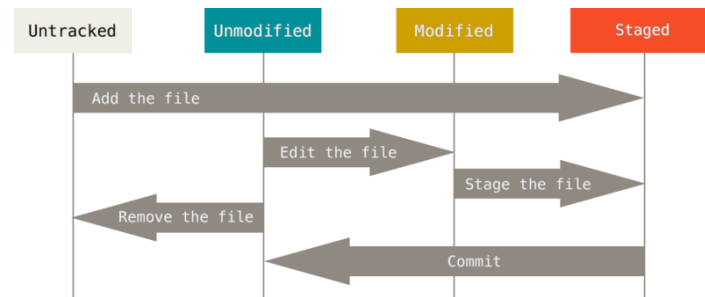


Figura 23: El ciclo de vida del estado de tus archivos.

A continuación, vas a realizar un cambio sobre el archivo `README.md`, registrarás los cambios en git y enviarás el nuevo *commit* al repositorio remoto:

1. Modifica y guarda el contenido de `README.md` para que contenga únicamente el título inicial y el enunciado usando un **editor de texto plano**.
2. Cambia la ubicación de la terminal sobre la ubicación de tu proyecto con el comando `cd taller00` y verifica el estado de tus archivos usando el comando `git status`. Ejemplo:

```

1 PS > cd taller00
2 PS > git status
3 On branch main
4 Your branch is up to date with 'origin/main'.
5
6 Changes not staged for commit:
7   (use "git add <file>..." to update what will be committed)
8   (use "git restore <file>..." to discard changes in working
   directory)
9       modified:   README.md
10
11 no changes added to commit (use "git add" and/or "git commit -a")

```

3. Prepara (*stage*) el archivo con `git add README.md` para un nuevo *commit*:

```

1 PS > git add README.md
2 PS > git status
3 On branch main
4 Your branch is up to date with 'origin/main'.
5
6 Changes to be committed:
7   (use "git restore --staged <file>..." to unstage)
8       modified:   README.md

```

Nota: Recuerda que puedes verificar el cambio del estado del archivo con `git status`.

4. Crea una nueva instantánea con el siguiente comando:

```
1 PS > git commit -m "solo enunciado"
2 [main 8f63fc1] R1 title
3 1 file changed, 2 insertions(+)
```

5. Usa el comando `git push` para enviar los cambios al repositorio remoto. Ejemplo:

```
1 PS > git push
2 Username for 'https://gitlab.com': g.cristancho01
3 Password for 'https://g.cristancho01@gitlab.com':
4 Enumerating objects: 5, done.
5 Counting objects: 100% (5/5), done.
6 Delta compression using up to 8 threads
7 Compressing objects: 100% (2/2), done.
8 Writing objects: 100% (3/3), 284 bytes | 284.00 KiB/s, done.
9 Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
10 To https://gitlab.com/is8003xx/taller00.git
11 d55b7da..8f63fc1 main -> main
```

8. Refinamiento 1

1. En el archivo `README.md` agrega un título de nivel 2, iniciando la línea con dos símbolos numeral (`##`), y luego sintetiza el problema en tus propias palabras haciendo énfasis en las entradas, salidas y transformación de la información[8, p. 114][9, p. 37].
2. Una vez consideres que haz extraído la información importante del enunciado original, **registra los cambios** realizados en git (tomar como referencia la sección de este documento).
3. Genera un *annotated tag*[10] sobre el *commit* que acabas de generar ejecutando el comando:

```
1 git tag -a r1 -m "refinamiento 1"
```

4. Ejecuta `git push` para enviar los cambios al repositorio remoto y luego envía la información relacionada a *tags* con `git push --tags`, ejemplo:

```
1 PS > git push --tags
2 Enumerating objects: 1, done.
3 Counting objects: 100% (1/1), done.
4 Writing objects: 100% (1/1), 171 bytes | 171.00 KiB/s, done.
5 Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
6 To https://gitlab.com/is8003xx/taller00.git
7 * [new tag]          r1 -> r1
```

9. Refinamiento 2

1. Al final del archivo `README.md` agrega un nuevo título de nivel 2 para tu segundo refinamiento. En esta etapa, enfatiza en la estructura general de la solución y en las variables que necesitarás para almacenar la información requerida. Un indicador de tarea completa es una relación “uno a uno” entre líneas de pseudo-código y el código final.
2. Agrega el tag `r2` con el comentario `refinamiento 2` (al igual en la sección anterior).

10. Código

Basado en el pseudo-código obtenido en el refinamiento 2, escribe tu solución del problema usando `C` en el archivo `calc.c`. No olvides agregar el tag `cod` sobre la versión final de tu código.

Referencia

- [1] G. Inc., «Unify the DevOps lifecycle with GitLab | GitLab». <https://about.gitlab.com/stages-devops-lifecycle/>, 2022.
- [2] G. Inc., «Groups | GitLab». <https://docs.gitlab.com/ee/user/group/index.html>, 2022.
- [3] G. Inc., «Members of a project | GitLab». <https://docs.gitlab.com/ee/user/project/members/>, 2022.
- [4] G. Inc., «Organize work with projects | GitLab». <https://docs.gitlab.com/ee/user/project/index.html>, 2022.
- [5] G. Inc., «Repository | GitLab». <https://docs.gitlab.com/ee/user/project/repository/index.html>, 2022.
- [6] S. Chacon y B. Straub, «Git Internals - Plumbing and Porcelain». <https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain>, 2022.
- [7] S. Chacon y B. Straub, «Git Basics - Recording Changes to the Repository». <https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository>, 2022.
- [8] P. Deitel y H. Deitel, *C How to Program, Global Edition*. Harlow, England: Pearson Education Limited, 2016.

- [9] L. J. Aguilar, *Programación En C, C++, Java y UML*. México D.F.: McGraw Hill, 2014.
- [10] S. Chacon y B. Straub, «2.6 Git Basics - Tagging». <https://git-scm.com/book/en/v2/Git-Basics-Tagging>, 2022.